

Progress Log — OCP Maintenance AI MVP

2026-02-20 — Session 1: Initialization & Blueprint

What was done

1. **Explored** full project directory structure (87 files, 8 subdirectories)
2. **Read and analyzed** all 9 SOFTWARE DEVELOPMENT CONTEXT markdown files:
 3. Antigravity Second Brain architecture
 4. Corporate Second Brain design (3 versions)
 5. Mining AI/digitalization strategy
 6. Neuro-architecture UX principles
 7. 10 AI prototype use cases (Reliability, Operations, Risk, HR, PMO, etc.)
 8. Megaproject software market research (18 solutions evaluated)
 9. AI startup vs incumbent competitive analysis
10. **Read and analyzed** both CLIENT CONTEXT PDFs:
 11. RFI presentation (17 slides): 3 MVP functionalities, tech stack, timeline, investment
 12. Data requirements document: 15 data categories, 3-tier priority, folder structure
13. **Created B.L.A.S.T. project scaffolding:**
 14. `architecture/` — Layer 1: SOPs (empty per protocol)
 15. `tools/` — Layer 3: Scripts (LOCKED until Blueprint approved)
 16. `.tmp/` — Temporary workbench
 17. `task_plan.md` — Phases, goals, checklists
 18. `findings.md` — Research, discoveries, constraints
 19. `progress.md` — This file
 20. `gemini.md` — Project Constitution
21. **Answered 5 Discovery Questions** (+ 2 follow-up):
 22. North Star: OCP MVP Pilot (3 functionalities)
 23. Integrations: SAP PM + Claude API
 24. Source of Truth: Generate phosphate-realistic synthetic data
 25. Delivery: Local prototype (Streamlit)
 26. Behavior: Safety-first AI (human validates everything)

27. Language: French + English + Arabic
28. Data fidelity: Phosphate-specific equipment
29. **Defined 9 JSON Data Schemas** in gemini.md:
30. Equipment Hierarchy, Field Capture Input, Structured Work Request
31. Planner Recommendation, Backlog Item, Optimized Backlog
32. Work Order History, Spare Parts/BOM, Maintenance Plans
33. **Codified Behavioral Rules** in gemini.md:
34. Safety-first AI rules, language rules, data rules, do-not rules
35. **Launched background research** for GitHub repos and resources

Errors

- None

Tests

- None yet (no code written — per protocol)

2026-02-20 — Session 2: R8 Deep Dive & Reference Document Architecture

What was done

1. **Read and analyzed ALL R8/Maintenance Strategy documents** (16 files):
2. R8 Software architecture: 4 areas (Sandbox, Component Library, Equipment Library, Plant Hierarchy)
3. Complete RCM methodology: 5-phase cyclic process
4. Criticality assessment: 11 consequence categories, 5 likelihood levels, 4 risk classes
5. Failure mode analysis: What + Mechanism + Cause, 6 Nowlan & Heap patterns
6. Strategy decision tree: Hidden/Evident → CBM/FT/RTF/FFI/Redesign
7. Task definition: naming conventions, acceptable limits, resources
8. Work packaging: Standalone/Suppressive/Sequential, grouping rules
9. Quality validation: 6-stage QA process, 40+ field-level rules
10. **Read and analyzed SAP Upload Templates** (3 files):
 - Maintenance Item: 18 fields, links to Task List via \$TL1
 - Task List: 25 fields, multi-operation format (auto electrician + fitter)
 - Work Plan: 21 fields, 14-day cycle with 50% call horizon
 - Cross-reference model: \$MI1/\$TL1 placeholder linking system
11. **Read and analyzed Work Instruction Templates** (Anglo American Coal):

- Cover page, safety section, resource summary, task list, completion section
- 4 WP type templates: Inspection Route, Service Package, ConMon Route, FFI Test

12. **Created 7 permanent reference documents** in architecture/:

- `ref-01-maintenance-strategy-methodology.md` — Full RCM process, decision trees
- `ref-02-r8-data-model-entities.md` — All entity schemas, 12+ entities, code tables
- `ref-03-sap-pm-integration.md` — SAP upload templates, field mappings
- `ref-04-quality-validation-rules.md` — 6-stage QA, 40+ validation rules, checklists
- `ref-05-client-context-ocp.md` — OCP pain points, MVP scope, timeline, metrics
- `ref-06-software-architecture-vision.md` — Second Brain vision, full tech stack, 10 use cases
- `ref-07-work-instruction-templates.md` — WI structure, WP type templates, naming conventions

13. **User requested new module:** Maintenance Strategy Development (4th MVP module)

14. **Expanded gemini.md:** Added 10 new schemas for Module 4 (§9.1–9.10):

- ComponentLibraryItem, EquipmentLibraryItem, PlantHierarchyNode
- CriticalityAssessment, Function, FunctionalFailure, FailureMode
- MaintenanceTask, WorkPackage, SAPUploadPackage

15. **Created additional reference documents:**

- `ref-08-user-guide-step-by-step.md` — Complete step-by-step for all 4 modules
- `ref-09-iso-55002-compliance-mapping.md` — ISO 55002 clause-by-clause mapping (73% compliance)

16. **Created BLUEPRINT.md** — Architecture diagram, build sequence, safety flow, file structure

17. **Created DOCUMENT_INDEX.md** — Master control register for all 48+ project documents

Errors

- None

Tests

- None yet (Blueprint phase)

2026-02-20 — Session 3: Full Code Implementation

What was done

Pydantic Data Models (19 schemas)

1. **Created** `tools/models/schemas.py` — All 19 Pydantic v2 models:

- `PlantHierarchyNode` (6-level: Plant → Area → System → Equipment → SubAssembly → MI)

- **CriticalityAssessment** (11 criteria × 5 probability, 4 risk classes)
- **Function** (PRIMARY/SECONDARY/PROTECTIVE)
- **FunctionalFailure** (TOTAL/PARTIAL)
- **FailureMode** (What + Mechanism + Cause, 18 mechanisms, 44 causes, 72 valid combos)
- **MaintenanceTask** (8 task types, 72-char name limit, labour + material resources)
- **WorkPackage** (40-char ALL CAPS name, 3 types: Standalone/Suppressive/Sequential)
- **SAPUploadPackage** (Maintenance Item + Task List + Work Plan)
- Plus 7 supporting models and 5 enums

Deterministic Engines (9 engines)

1. **Created** `tools/engines/criticality_engine.py` — 11 consequence categories, 4 risk classes, score calculation
2. **Created** `tools/engines/rcm_decision_engine.py` — 16-path deterministic RCM decision tree:
 - Hidden/Evident classification
 - Safety/Environmental/Operational/Non-operational consequence routing
 - CBM/FT/RTF/FFI/Redesign strategy selection
 - Calendar vs Operational cause logic for frequency recommendations
 - Full audit trail with path tracing
3. **Created** `tools/engines/sap_export_engine.py` — Complete SAP PM upload generation:
 - Maintenance Item (18 fields, \$MI refs)
 - Task List (25 fields, multi-operation)
 - Work Plan (21 fields, 14-day cycle, 50% call horizon)
 - Cross-reference linking (\$MI1 ↔ \$TL1)
4. **Created** `tools/engines/priority_engine.py` — Priority calculation based on criticality + age + safety
5. **Created** `tools/engines/backlog_grouper.py` — Work package grouping:
 - 5 grouping strategies: frequency, constraint, trade, equipment, shutdown
 - Stratification by blocking reason, priority, criticality
 - Schedule optimization
6. **Created** `tools/engines/equipment_resolver.py` — Fuzzy matching for equipment identification
7. **Created** `tools/engines/material_mapper.py` — Material assignment:
 - T-16 rule: REPLACE tasks → require materials
 - BOM lookup, stock validation, criticality-based suggestions
8. **Created** `tools/engines/work_instruction_generator.py` — 4 WI templates:
 - Inspection Route, Service Package, ConMon Route, FFI Test
 - Cover page, safety section, resource summary, task sequence
9. **Created** `tools/engines/state_machine.py` — 6 entity workflows:

- DRAFT → IN_REVIEW → APPROVED → EXPORTED
- Transition validation, audit trail logging

Validators (3 validators)

1. **Created** `tools/validators/quality_validator.py` — 40+ quality validation rules:
 - Failure mode rules (mechanism+cause validation, naming, patterns)
 - Task rules (naming conventions, acceptable limits, frequency, resources)
 - Work package rules (naming format, labour summary, grouping rules)
 - SAP rules (field mapping, cross-reference integrity)
2. **Created** `tools/validators/confidence_validator.py` — AI confidence scoring:
 - Per-field confidence thresholds
 - Batch validation across entity collections
3. **Created** `tools/validators/naming_validator.py` — Naming convention enforcement:
 - Task names: 72 chars, "Inspect [what] for [evidence]" pattern
 - WP names: 40 chars, ALL CAPS, [FREQ] [ASSET] [TRADE] [TYPE] [CONSTRAINT]
 - FM "what" field: capital letter, singular noun

Generator

1. **Created** `tools/generators/synthetic_data.py` — Phosphate-realistic synthetic data:
 - Equipment hierarchy (SAG Mill, Belt Conveyor, Crusher, Flotation Cell, etc.)
 - Failure modes with valid 72-combo Mechanism+Cause pairs
 - Maintenance tasks with proper naming conventions
 - Work packages with correct grouping

Test Suite (282 tests)

1. **Created 16 test files** covering all engines, validators, generators, and models:
 - `tests/conftest.py` — Shared fixtures (equipment, failure modes, tasks, work packages)
 - `tests/test_schemas.py` — 19 model validation tests
 - `tests/test_criticality.py` — Criticality engine tests
 - `tests/test_rcm_decision_tree.py` — All 16 RCM paths tested
 - `tests/test_sap_upload.py` — SAP export generation tests
 - `tests/test_state_machine.py` — State transition tests
 - `tests/test_equipment_resolver.py` — Fuzzy matching tests
 - `tests/test_priority_engine.py` — Priority calculation tests
 - `tests/test_backlog_grouper.py` — Grouping strategy tests
 - `tests/test_material_mapper.py` — Material assignment tests

- `tests/test_work_instruction.py` — WI generation tests
- `tests/test_confidence.py` — Confidence validation tests
- `tests/test_quality_rules.py` — 40+ quality rule tests
- `tests/test_naming_conventions.py` — Naming pattern tests
- `tests/test_synthetic_data.py` — Data generator tests
- `tests/test_internationalization.py` — Trilingual support tests
- **Result: 282 tests PASS**

Errors

- None (all 282 tests green)

Deliverables

- 19 Pydantic models in `tools/models/schemas.py`
- 9 engines in `tools/engines/`
- 3 validators in `tools/validators/`
- 1 generator in `tools/generators/`
- 16 test files, 282 tests passing
- `pytest.ini` configuration
- `requirements.txt` (pydantic>=2.0, pytest>=8.0, python-dateutil>=2.8)

2026-02-20 — Session 4: GECAMIN Competitive Intelligence & Strategic Analysis

What was done

GECAMIN MAPLA 2024 Cross-Reference (REF-10)

1. **Analyzed 22+ GECAMIN conference presentations** from 57 total PDFs
2. **Created** `architecture/ref-10-gecamin-cross-reference.md` — Complete cross-reference:
 - **myRIAM SYSTEM (Guayacán Solutions)** — Direct competitor analysis:
 - Feature-by-feature comparison across 12 dimensions
 - Their gaps: no SAP integration, no QA rules, no UX differentiation, no ISO compliance
 - Our gaps: predictive ML models (Phase 2), mobile app, conversational interface
 - **6 Thematic Categories** mapped to our modules:
 - AI for Condition Monitoring: 6 presentations (55-83% efficiency gains validated)
 - AI Chatbots: SQM Copilot Studio chatbot validates our M2 concept

- Digital Twins: 3 presentations validate Phase 3-4 vision
- Maintenance Optimization: 6 presentations validate M3/M4 approach
- Generative AI in Mining: Centinela validates our M1 approach
- Enterprise Asset Management: 5 presentations validate scale feasibility
- **Competitive Positioning Matrix** — Green/Yellow/Red analysis
- **9 Validated Claims** with independent GECAMIN evidence
- **9 Recommendations** (immediate, Phase 2, Phase 3-4)

Neuro-Arquitectura Integrated Review (REF-11)

1. **Created** [architecture/ref-11-neuro-arquitectura-review.md](#) — 6 behavioral science pillars:
 - **Pillar 1: Shared Mental Models (SMM)** — Mapped to all 4 modules
 - **Pillar 2: Transactive Memory Systems (TMS)** — Expert Cards, resolution history
 - **Pillar 3: Self-Determination Theory (SDT)** — Autonomy, Competence, Relatedness
 - **Pillar 4: Cognitive Load Theory (CLT)** — Intrinsic/Extraneous/Germane load management
 - **Pillar 5: Psychological Safety** — Non-punitive language, draft modes, ipsative feedback
 - **Pillar 6: Behavioral Nudges** — 7 nudge types with specific implementations
 - **Bias Mitigation** — Hidden Profile, Confirmation, Anchoring bias countermeasures
 - **ISO 55002 Integration** — 6 ISO gaps addressed by Neuro-Architecture
 - **GECAMIN Cross-Validation** — 5 GECAMIN findings mapped to Neuro-Architecture responses
 - **Implementation Checklists** — Per-module checklist (20+ items)

Final Strategic Recommendations (REF-12)

1. **Created** [architecture/ref-12-strategic-recommendations.md](#) — 10 strategic recommendations:
 - **Rec 1 (CRITICAL):** Lead with "Cognitive Prosthesis" narrative
 - **Rec 2 (HIGH):** SAP Integration as primary technical moat
 - **Rec 3 (HIGH):** "Enhance, Don't Replace" AI model — formalized interaction table
 - **Rec 4 (HIGH):** Asset Health Index model (CriticalityScore + BacklogPressure + StrategyCoverage + ConditionStatus + ExecutionCompliance)
 - **Rec 5 (MEDIUM):** Conversational interface (Phase 2 roadmap)
 - **Rec 6 (HIGH):** Predictive maintenance capability (Weibull, NHPP statistical methods)
 - **Rec 7 (HIGH):** Multi-plant scalability (per-plant config, variance alerts)
 - **Rec 8 (MEDIUM):** ISO 55002 strategic compliance (PEGA, KPI Dashboard, CAPA, Stakeholder, Management Review)
 - **Rec 9 (CRITICAL):** Phase 0 structured plan (4-week activity schedule)
 - **Rec 10 (STRATEGIC):** 3-Layer competitive moat (Domain Knowledge + Behavioral Science + Data Network Effects)
 - **Implementation Priority Matrix** — Must Have / Should Have / Nice to Have

- **Updated Risk Assessment** — 6 risks with GECAMIN-sourced mitigations

DOCUMENT_INDEX Update

1. **Updated DOCUMENT_INDEX.md** with:
 - REF-10, REF-11, REF-12 in Tier 2 (Competitive Intelligence & Strategy)
 - SRC-70 to SRC-82: 13 GECAMIN source documents indexed
 - 17 new routing table entries
 - Document count: 12 summaries, 48+ sources

Errors

- None

Tests

- 282 tests still passing (no code changes in this session)

2026-02-20 — Session 5: FM Table Integration, Cross-References & Multi-Agent Architecture Plan

What was done

Failure Modes Lookup Table Integration (SRC-09)

1. **Integrated the authoritative FM lookup table** from `Failure Modes (Mechanism + Cause).xlsx` :
 - 72 valid Mechanism+Cause combinations across 18 mechanisms
 - Updated `tools/models/schemas.py` :
 - `FailureMechanism` enum: 9 → 18 values (added Arcs, Blocks, Drifts, Expires, etc.)
 - `FailureCause` enum: 18 → 44 values (added 26 new causes)
 - `VALID_FM_COMBINATIONS` : dict mapping each mechanism to its valid causes
 - Added Pydantic `@model_validator` on `FailureMode` to enforce 72-combo constraint
 - Updated `gemini.md` §4.4: New "Failure Mode Rules" section (6 rules)
 - Updated `gemini.md` §4.5: New do-not rule #6 for FM validation
 - Updated `DOCUMENT_INDEX.md` : Added SRC-09 as MANDATORY source, added routing entries

Cross-Reference Updates (REF-01 & REF-02)

1. **Updated** `architecture/ref-01-maintenance-strategy-methodology.md` §3.3:
 - Replaced old 9-mechanism table with authoritative 18 mechanisms (with descriptions)

- Replaced old 18-cause table with authoritative 44 causes (with category annotations)
 - Added MANDATORY cross-reference notice to SRC-09
2. **Updated** `architecture/ref-02-r8-data-model-entities.md` :
 - §2.6: FailureMode entity fields now reference SRC-09
 - §3.1: Cause codes expanded from 18 → 44 with MANDATORY SRC-09 reference
 - §3.2: Mechanism codes expanded from 9 → 18 with MANDATORY SRC-09 reference

Test Suite Maintenance

1. **Updated test fixtures** (`tests/conftest.py`) — All fixtures use valid 72-combo FM pairs
2. **Updated** `tests/test_schemas.py` — Added FM validation tests for valid/invalid combos
3. **Updated** `tests/test_rcm_decision_tree.py` — All 16 paths use valid FM combos
4. **Full test suite: 282 tests PASS** (zero regressions)

Multi-Agent Architecture Plan (Approved)

1. **Designed multi-agent architecture** for AI-driven R8 methodology execution:
 - **Orchestrator Agent** (sonnet) — Coordinates 4-milestone workflow
 - **Reliability Agent** (opus) — FMEA, RCM analysis, criticality
 - **Planning Agent** (sonnet) — Work packaging, SAP export, work instructions
 - **Spare Parts Agent** (haiku) — Material assignment, BOM lookup
 - **4 Milestone Gates**: Hierarchy → FMEA → Strategy+Tasks → SAP Upload
 - Human approves at each gate; agents perform all analysis
2. **Created directory structure**:
 - `agents/__init__.py` — Multi-Agent System root
 - `agents/tool_wrappers/__init__.py` — MCP tool wrappers
 - `agents/definitions/__init__.py` — Agent definitions
 - `agents/definitions/prompts/` — System prompt directory
 - `agents/orchestration/__init__.py` — Orchestration layer
3. **Updated dependencies**:
 - `requirements.txt` : Added `anthropic>=0.39.0` , `pytest-asyncio>=0.23.0`
 - Installed: anthropic 0.83.0, pytest-asyncio 1.3.0

Errors

- Minor: Windows encoding display issue with `pip show` output (cosmetic only, packages installed correctly)

Tests

- 282 tests PASS

Status at End of Session 5

- **Plan approved** for multi-agent implementation
 - **Directory structure** created but no agent code written yet
 - **Next:** Implement all GECAMIN-identified improvements, then continue multi-agent plan
-
-

2026-02-20 — Sessions 6-8: Full Stack Build (Phase 2 → Phase 8)

What was done

(Sessions 6 through 8 built the complete application stack. Key milestones:)

1. **Phase 2 — Application Layer:** FastAPI backend (17 routers, 21 services), SQLAlchemy ORM (28 models), Streamlit frontend (17 pages), full HTTP API client
2. **Phase 3 — Engine Expansion:** 36 deterministic engines (up from 9), 62+ MCP tool wrappers, 4 AI agents with system prompts
3. **Phase 4A — GFSN Alignment:** Configurable criticality bands, SAP lifecycle states, RCA engine, Planning KPIs (11), DE KPIs (5)
4. **Phase 4B — Scheduling:** Weekly program engine, resource leveler, Gantt generator, shutdown integration
5. **Phase 5 — Advanced Reliability:** Spare parts (VED/FSN/ABC), MoC workflow, OCR, Jack-Knife, LCC, Pareto, RBI
6. **Phase 6 — Reporting & Dashboards:** Report generation, notifications, cross-module analysis, executive dashboard
7. **Phase 7 — FMECA:** 4-stage worksheets, RPN calculation, FMECA API + UI
8. **Phase 8 — RCA/DE Full Stack:** RCA models, Planning/DE KPI snapshots, 9 RCA endpoints, Defect Elimination page

Tests

- **1099 tests passing** across 78 test files
-

2026-02-21 — Session 9: Refinement & Polish (Phase 9)

What was done

1. **Trilingual UI:** French/English/Arabic language switcher across all 17 Streamlit pages
2. **i18n System:** JSON translation files (en.json, fr.json, ar.json) with ~350 keys each
3. **Professional Styling:** OCP green theme, charts, KPI visualizations (Plotly)
4. **User Feedback:** Feedback collection page with API integration
5. **Integration Testing:** End-to-end test coverage

Tests

- **1099 tests passing** (all green)
-

2026-02-22 — Session 9B: Data Templates, Documentation, Equipment Libraries & Hierarchy Builder

What was done

D1: Excel Templates for Data Loading (13 templates)

1. **Created** `templates/generate_templates.py` (~750 lines):
2. 13 .xlsx files with OCP green headers (#1B5E20), frozen panes, data validation dropdowns
3. 3-5 realistic phosphate example rows per template
4. Template 03 includes "Valid FM Combinations" sheet with all 72 combos
5. Template 04 uses multi-sheet relational design:
 - Tasks (task_id PK), Task_Labour (labour_id PK, task_id FK, worker_id, company),
 - Task_Materials (material_line_id PK, task_id FK, equipment_bom_ref),
 - Task_Tools (tool_line_id PK, task_id FK, tool_code)
6. Templates: equipment_hierarchy, criticality_assessment, failure_modes, maintenance_tasks, work_packages, work_order_history, spare_parts_inventory, shutdown_calendar, workforce, field_capture, rca_events, planning_kpi_input, de_kpi_input
7. **Tests:** 55 tests in `tests/test_templates.py` (all passing)

D2: Tool Listing Documentation

1. **Created** `docs/tool-listing.md` — Comprehensive listing of all 36 engines, 62+ MCP tools, 3 validators, 5 processors, 1 generator

D3: Variable Listing Documentation

1. **Created** `docs/variable-listing.md` — All enums (~70), database fields (30 tables), API fields (17 routers), KPI definitions, 72 valid FM combinations

D4: Software Glossary

1. **Created** `docs/glossary.md` — Trilingual (EN/FR/AR) glossary: ~105 terms across 4 domains (Maintenance Methodology, OCP/Phosphate, SAP PM, Software-Specific)

D5: How-To Guides (6 user role guides)

1. **Created** `docs/how-to-guides/` — 6 step-by-step guides:
2. 01-field-technician.md (FT-01/FT-02)
3. 02-maintenance-planner.md (MP-01 to MP-05)
4. 03-reliability-engineer.md (RE-01 to RE-07)
5. 04-analyst.md (AN-01 to AN-09)
6. 05-administrator.md (AD-01 to AD-05)
7. 06-manager.md (MG-01 to MG-05)

D6: Hierarchy Builder from Vendor Data

1. **Created** `tools/engines/hierarchy_builder_engine.py` (~250 lines):
2. `build_from_vendor()` — Builds Equipment → Sub-Assembly → MI hierarchy tree
3. `auto_assign_criticality()` — Power/type-based criticality assignment
4. `generate_standard_failure_modes()` — FM generation validated against 72-combo
5. `generate_tag()` — OCP tag convention (AREA-TYPE-SEQ)
6. Loads from `equipment_library.json` with alias matching
7. **Created** `api/services/hierarchy_builder_service.py` — Service layer (persist to DB, audit log)
8. **Modified** `api/routers/hierarchy.py` — Added `POST /api/v1/hierarchy/build-from-vendor`
9. **Created** `agents/tool_wrappers/hierarchy_builder_tools.py` — 3 MCP tools (`build_hierarchy_from_vendor`, `get_equipment_types`, `auto_assign_criticality`)
10. **Modified** `agents/tool_wrappers/server.py` — Registered 3 new tools → **126 MCP tools total**
11. **Modified** `streamlit_app/pages/1_hierarchy.py` — "Build from Vendor Data" form with equipment type dropdown
12. **Updated i18n** — 9 new keys in `en.json`, `fr.json`, `ar.json`
13. **Tests:** 24 tests in `tests/test_hierarchy_builder.py` (all passing)

D7: Equipment Type Libraries

1. **Created** `Libraries/equipment_library.json` (~1500 lines) — 15 phosphate equipment types (SAG Mill, Ball Mill, Slurry Pump, Flotation Cell, Belt Conveyor, etc.) with sub-assemblies, MIs, failure modes, Weibull params
2. **Created** `Libraries/component_library.json` (~800 lines) — Standard components (bearings, seals, motors, gearboxes, impellers, etc.) with failure mechanisms and Weibull params
3. **Created** `Libraries/README.md` — Library documentation
4. **Tests:** 30 tests in `tests/test_equipment_library.py` (all passing)

Errors & Fixes

- Template header count assertion (13→14 for criticality assessment)
- Invalid FM combo in example data (WEARS+ABRASION → SEVERS+ABRASION)
- Equipment library JSON key mapping (equipment_type_id, criticality_class, weibull_eta)
- SQLite PRAGMA pattern in test fixtures (dbapi_conn.cursor().execute)
- Tool count assertion (123→126 after adding hierarchy builder tools)

Tests

- **1208 tests passing** (1099 existing + 55 template + 30 equipment library + 24 hierarchy builder)
-

Cumulative Project Statistics

Category	Count
Reference documents (REF-01 to REF-12)	12
Source documents indexed (SRC-xx)	48+
Pydantic data models	30+
Deterministic engines	37 (36 + hierarchy builder)
Validators	3
Generators	1
Processors	5
MCP tool wrappers	126
AI agents	4
API routers	17
API services	22
Streamlit pages	17
Excel templates	13
Equipment types in library	15
Test files	82
Tests passing	1208
Valid FM combinations	72 (18 mechanisms × variable causes)
i18n translation keys	~360 per language (EN/FR/AR)
Documentation files	12 (tool listing, variables, glossary, 6 how-to guides, data README, 2 library files)

Architecture State

```
ASSET-MANAGEMENT-SOFTWARE/  
├─ architecture/           # 12 reference docs + Blueprint + DOCUMENT_INDEX  
├─ tools/  
│   ├── models/schemas.py # 30+ Pydantic models (72-combo FM enforced)  
│   ├── engines/           # 37 deterministic engines  
│   ├── validators/        # 3 validators (quality, confidence, naming)  
│   └─ generators/         # 1 synthetic data generator  
├─ agents/  
│   ├── tool_wrappers/     # 28+ tool wrapper files  
│   ├── definitions/       # Agent definitions + prompts  
│   └─ orchestration/      # Workflow coordination  
├─ api/  
│   ├── routers/          # 17 API routers  
│   ├── services/         # 22 service modules  
│   └─ database/          # SQLAlchemy ORM (28+ models)  
├─ streamlit_app/  
│   ├── pages/            # 17 feature pages  
│   └─ i18n/              # EN/FR/AR translation files  
├─ templates/             # 13 Excel data-loading templates  
├─ data/                  # Equipment + component libraries (JSON)  
├─ docs/  
├─ tests/                 # 82 test files, 1208 tests passing  
├─ gemini.md              # Project Constitution  
├─ DOCUMENT_INDEX.md      # Master navigation  
├─ requirements.txt        # All dependencies  
└─ pytest.ini             # Test configuration
```